Rockchip RK3572 USB 开发指南

文件标识：RK-SM-YF-C67

发布版本：V1.1.0

日期：2026-05-13

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有© 2026瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档提供 RK3572 USB 模块的开发指南，目的是让开发者理解 RK3572 USB 控制器和 PHY 的硬件电路设计以及对应的软件 DTS 配置，以便开发者根据产品的 USB 应用需求进行灵活设计和快速开发。

芯片名称	内核版本
RK3572	Linux-6.12

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

硬件开发工程师

修订记录

日期	版本	作者	修改说明
2026-04-22	V1.0.0	吴良峰 王明成	初始版本
2026-05-13	V1.1.0	吴良峰	修正 RK809 Type-C 控制器芯片名称

目录

Rockchip RK3572 USB 开发指南

1. RK3572 USB 控制器和 PHY 简介
2. RK3572 USB 硬件电路设计
   - 2.1 USB 控制器供电及功耗管理
   - 2.2 USB PHY 供电及功耗管理
     - 2.2.1 USB 2.0 PHY 供电及功耗管理
     - 2.2.2 USB 3.0 PHY 供电及功耗管理
   - 2.3 USB 硬件电路设计注意点
     - 2.3.1 USB_DRD0_VBUSDET 电路设计
     - 2.3.2 Maskrom USB 电路设计
     - 2.3.3 Type-C USB 3.0 硬件电路
       - 2.3.3.1 Type-C USB 3.0 Switch 硬件电路设计
       - 2.3.3.2 Type-C USB 2.0 Switch 硬件电路设计
     - 2.3.4 Type-C USB 2.0 with PD 硬件电路
     - 2.3.5 Type-C USB 2.0 only 硬件电路
     - 2.3.6 Type-A USB 3.0 硬件电路
     - 2.3.7 Multi Type-A USB 3.0 硬件电路
3. RK3572 USB DTS 配置
   - 3.1 USB 芯片级 DTSI 配置
     - 3.1.1 USB 控制器 DTSI 配置
     - 3.1.2 USB PHY DTSI 配置
   - 3.2 Type-C USB 3.0 Switch 全功能 DTS 配置
   - 3.3 Type-C USB 2.0 Switch 全功能 DTS 配置
   - 3.4 Type-C USB 2.0 with PD DTS 配置
   - 3.5 Type-C USB 2.0 only DTS 配置
   - 3.6 Type-A USB 3.0 DTS 配置
   - 3.7 Multi Type-A USB 3.0 DTS 配置
   - 3.8 USB3.0 Combo PHY 不供电的 DTS 配置
     - 3.8.1 Combo PHY0 不供电的 DTS 配置
     - 3.8.2 Combo PHY1 不供电的 DTS 配置
   - 3.9 USB DTS 重要属性配置说明
     - 3.9.1 USB 控制器 DTS 属性配置说明
     - 3.9.2 USB 2.0 PHY DTS 属性配置说明
     - 3.9.3 USB 3.0 Combo PHY DTS 属性配置说明
     - 3.9.4 Android GKI 依赖属性配置
4. RK3572 USB DRD mode 切换命令
5. RK3572 USB 下载固件的功能配置
   - 5.1 Rockusb3.0 下载工具配置
     - 5.1.1 Windows 图形界面工具配置
     - 5.1.2 Linux 命令行工具配置
     - 5.1.3 Mac 命令行工具配置
   - 5.2 Rockusb2.0 下载工具配置
   - 5.3 Rockusb2.0 Kernel/U-Boot 配置
6. RK3572 USB 2.0 低功耗唤醒系统的功能配置
   - 6.1 RK3572 USB 2.0 唤醒功能描述
   - 6.2 RK3572 USB 2.0 唤醒功能的硬件电路设计要求
   - 6.3 RK3572 USB 2.0 唤醒功能的软件配置
7. RK3572 Type-C 控制器芯片支持列表
8. 参考文档

1. RK3572 USB 控制器和 PHY 简介

RK3572 支持 2 个 独立的 USB 3.0 DRD 控制器、2 个 独立的 USB 2.0 PHY、2 个 USB3.0/SATA/PCIe Combo PHY。如果要了解更详细的 USB 控制器特性，请参阅 RK3572 TRM。

RK3572 USB 的 CCI/MMU/DEBUG 功能设计如下：

1. USB DRD0/1 默认都支持 CCI 保证 Cache 一致性，即不需要 CPU 执行刷 cache 的操作；
2. USB DRD0/1 都不支持 MMU，即不允许 USB 控制器硬件直接访问超过 4G 的内存空间；
3. USB DRD0 DP/DM 支持与 UART/JTAG 复用，SDK 默认使能该功能，需要结合 RK USB to DEBUG 转接板使用。

表 1 RK3572 USB 控制器和 PHY 的连接关系

USB 接口名称(原理图)	**USB 控制器**	**USB PHY**
USB DRD0	USB0 (DWC3&xHCI)	USB3.0/SATA/PCIe ComboPHY0 + USB2.0 PHY0
USB DRD1	USB1 (DWC3&xHCI)	USB3.0/SATA/PCIe ComboPHY1 + USB2.0 PHY1

Note:

1. USB 3.0 与 USB 3.1 Gen1 物理层传输速率一样为 5Gbps，USB 2.0 物理层传输速率为 480Mbps;
2. USB3.0/SATA/PCIe Combo PHY 只支持 1 组 Tx/Rx，如果要支持 Type-C USB 3.0 的正反面硬件电路设计，需要增加 USB Switch 外围芯片；
3. USB3.0/SATA/PCIe Combo PHY 在同一时刻，只能支持一种工作模式，也即 USB 3.0 与 SATA/PCIe 接口是互斥的。

RK3572 USB 控制器和芯片端 USB 传输数据的 pin 脚的对应关系如下表 2 所示。

表 2 RK3572 USB 接口和 USB Pin 的对应关系

USB接口	**USB 2.0 Pin**	**USB 3.0 Pin**	**VBUSDET**	**ID**
USB DRD0	USB_DRD0_DP/USB_DRD0_DM	USB_DRD0_SSTXP/USB_DRD0_SSTXN USB_DRD0_SSRXP/USB_DRD0_SSRXN	USB_DRD0_VBUSDET	USB_DRD0_ID
USB DRD1	USB_DRD1_DP/USB_DRD1_DM	USB_DRD1_SSTXP/USB_DRD1_SSTXN USB_DRD1_SSRXP/USB_DRD1_SSRXN	USB_DRD1_VBUSDET	USB_DRD1_ID

Note:

1. USB DRD0/1 都支持 DRD mode，可以动态切换 Device 和 Host Mode，但 Device 和 Host 功能不能同时使用；
2. USB Host 的 USB 2.0 Pin 和 USB 3.0 Pin 可以独立同时使用；
3. USB Device 的 USB 2.0 Pin 和 USB 3.0 Pin 不支持独立同时使用；
4. 支持 USB3.0 P/N lane polarity inversion；
5. USB_DRD0_VBUSDET 用于 Device mode 检测连接和断开，对于带外置 Type-C 控制器芯片的 Type-C 接口，建议 USB_DRD0_VBUSDET 固定上拉到 VCC_3V3_S0，对于其他没有带外置 Type-C 控制器芯片的电路设计方案，要求 USB_DRD0_VBUSDET 仍然按照传统的分压电路设计，连接到 USB 接口的 VBUS 引脚，不作常供电的设计，也不能悬空；

6. USB_DRD1_VBUSDET 对应的 USB DRD1 如果有支持 Device mode 的需求，参考第 5 点设计。如果只有支持 Host mode 的需求，USB_DRD1_VBUSDET 可以悬空或者固定上拉到 VCC_3V3_S0；
7. USB_DRD0_ID/USB_DRD1_ID 只用于 Micro 接口类型的 OTG 功能，如果产品未使用 ID 脚，悬空即可；
8. USB 3.0 TXP/TXN 串联的电容建议为 100nF（USB 3.0 Spec 规定 AC Coupling Capacitor 75 ~ 200nF），电容值不满足要求可能导致 USB 3.0 链路训练和数据通信异常；
9. USB 2.0 DP/DM 建议串联 2.2 Ω电阻。

RK3572 USB 控制器和 PHY 的内部连接关系，以及对应的常见 USB 物理接口如下图 1 所示。

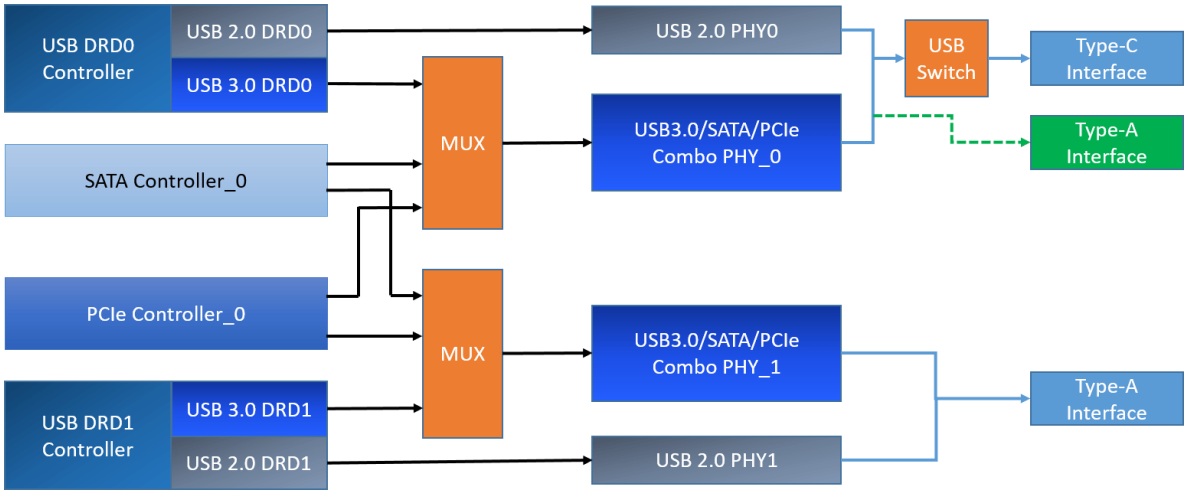


图 1 RK3572 USB 控制器和 PHY 的连接示意图

Note:

1. RK3572 USB 支持的接口类型并不局限于图 1 所描述的 Type-C/A USB 接口类型，还可以支持所有常见的 USB 接口，包括 Type-C USB 2.0/3.0，Type-A USB 2.0/3.0，Micro USB 2.0/3.0 等。为了适配不同的 USB 电路设计和接口类型，Linux-6.12 内核 USB 驱动已经做了软件兼容，开发者只需要根据产品的 USB 硬件电路，对 Linux USB DTS 进行正确配置，即可使能对应的 USB 接口功能。详细的 USB DTS 配置方法，请参考 RK3572 USB DTS 配置；
2. USB 3.0 与 SATA/PCIe 接口是互斥的，如果 Combo PHY0/1 用于 SATA/PCIe 接口，那么对应的 USB 控制器只能支持 USB 2.0。

2. RK3572 USB 硬件电路设计

2.1 USB 控制器供电及功耗管理

RK3572 USB 控制器的供电电源是 VD_LOGIC。同时，芯片内部有设计 USB 控制器专用的 power domain：RK3572_PD_PHP。

在实际使用场景中，Linux USB 控制器驱动会根据 USB 接口的工作情况，基于 Linux PM Runtime 机制，动态开关 USB 控制器的 PD，以降低 USB 控制器的功耗。而当系统进入二级待机时，为了达到最优功耗的目的，软件会强制关闭 USB 控制器的 PD，待机唤醒后，USB 控制器驱动会重新初始化，同时，也会重新枚举连接在 USB Host 接口上的 USB 外设。在实际产品的应用场景中，如果需要在二级待机唤醒过程中，保持 USB 的连接状态，请参考 Redmine USB 开发资料：
https://redmine.rock-chips.com/documents/113

USB 控制器的功耗控制策略如下：

1. 对于不使用的 USB 控制器，建议将对应的控制器 DTS 节点配置为 disabled；

2. 对于内核已启用的 USB 控制器，当配置为 Host mode 时，内核 USB 驱动已经支持 USB 控制器 Auto suspend 功能 (当 USB HOST 接口不接任何外设时，控制器自动进入 suspend 低功耗状态)，因此，开发者不需要对 USB 控制器的动态功耗管理进行调试；
3. 对于内核已启用的 USB 控制器，当配置为 DRD mode 时，内核 USB 驱动支持 PM Runtime 机制，可根据 USB 接口的连接情况动态开关 USB 控制器的 PD。

2.2 USB PHY 供电及功耗管理

2.2.1 USB 2.0 PHY 供电及功耗管理

RK3572 支持 2 个 独立的 USB 2.0 PHY0/1。在芯片内部，所有 USB 2.0 PHY 都属于 VD_USBPHY，同时，2 个 USB 2.0 PHY 共用如下图 2 所示的 3 路外部供电电源。因此，在系统运行时，无法通过硬件断电和关闭 PD 的简单方法，来降低 USB 2.0 PHY 的功耗。

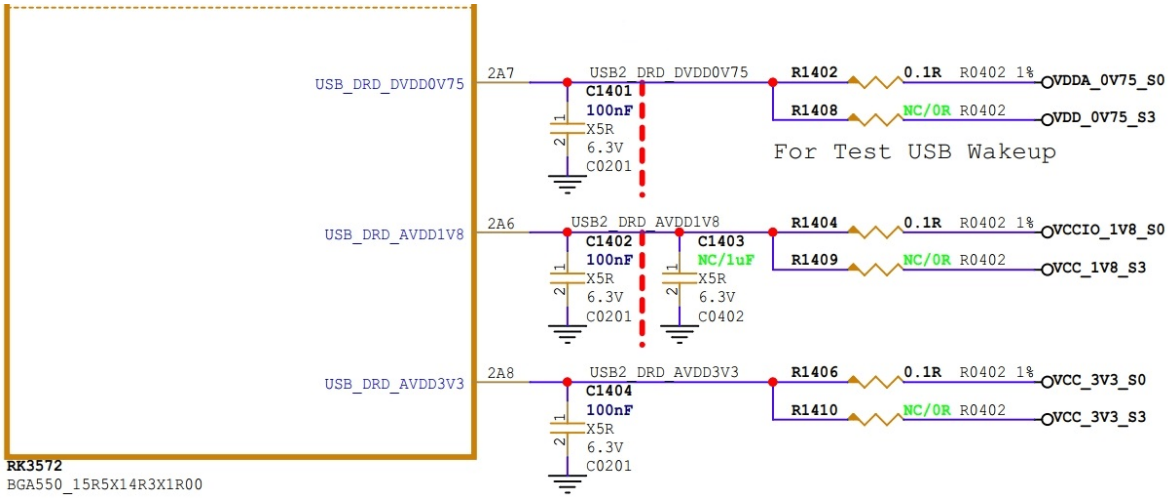


图 2 USB 2.0 PHY 供电电源

表 3 USB 2.0 PHY 供电电压要求

供电电源	最小	正常	最大	Unit
USB_DRD_DVDD0V75	0.6975	0.75	0.825	V
USB_DRD_AVDD1V8	1.674	1.8	1.98	V
USB_DRD_AVDD3V3	3.069	3.3	3.63	V

Note:

1. 在实际电路中，USB 2.0 PHY 的供电电压值不能超过正常值的 ±10%，如果超过规定的最大值或者低于规定的最小值，都可能会导致 USB 通信异常；
2. 如果要支持 USB 唤醒系统的功能，需要将 USB 2.0 PHY 相关的三路电源接合并到 PMU 的供电电源 VDD_0V75_S3/VCC_1V8_S3/VCC_3V3_S3，以保证二级待机时 USB 2.0 PHY 保持供电。

USB 2.0 PHY 的功耗控制策略如下：

1. 为了支持 Maskrom USB 下载固件的功能，必须保证 USB 2.0 PHY 的三路供电均正常；
2. 系统上电后，USB 2.0 PHY0/PHY1 默认处于 Normal mode。在 U-Boot SPL 阶段，软件会主动配置 USB 2.0 PHY1 处于最低功耗 IDDQ mode（PHY0 用于下载固件，SPL 不做低功耗配置）。在进入系统后，内核 USB 驱动会根据应用需求，设置对应的 USB 2.0 PHY1 退出 IDDQ mode；

3. 对于内核已启用的 USB 2.0 PHY，内核 USB 2.0 PHY 驱动会自动对 PHY 进行动态功耗控制，当检测到 USB 连接时，自动设置 USB 2.0 PHY 处于 Normal mode，当检测到 USB 断开时，自动设置 USB 2.0 PHY 处于 Suspend mode。

USB 2.0 PHY 处于不同工作模式的功耗数据如下表 4 所示。

表 4 USB 2.0 PHY 功耗数据（统计为单个 USB 2.0 PHY 的功耗）

供电电源	读写数据	动态休眠	PHY disabled	二级待机	Unit
USB_DRD_DVDD0V75	6.1	2.8	0.16	0	mA
USB_DRD_AVDD1V8	5.1	0.10	0.10	0	mA
USB_DRD_AVDD3V3	2.2	0.12	0.12	0	mA

Note:

1. 读写数据功耗的测试场景：接 U 盘拷贝数据，PHY 处于 Normal mode；
2. 动态休眠功耗的测试场景：USB 2.0 PHY 的 DTS enable，但不接 USB 外设，PHY 处于 Suspend mode；
3. PHY disabled 功耗的测试场景：USB 2.0 PHY 的 DTS disabled，PHY 处于 IDDQ mode；
4. 二级待机功耗的测试场景：USB 2.0 PHY 的三路供电电源全部关闭。

2.2.2 USB 3.0 PHY 供电及功耗管理

RK3572 支持 2 个独立的 USB3.0/SATA/PCIe Combo PHY0/1 。在芯片内部，PHY0/1 均属于 PD_BUS (Alive)，无动态 Powerdown 功能。在芯片外部，PHY0/1 有独立的供电电源，如图 3 所示。

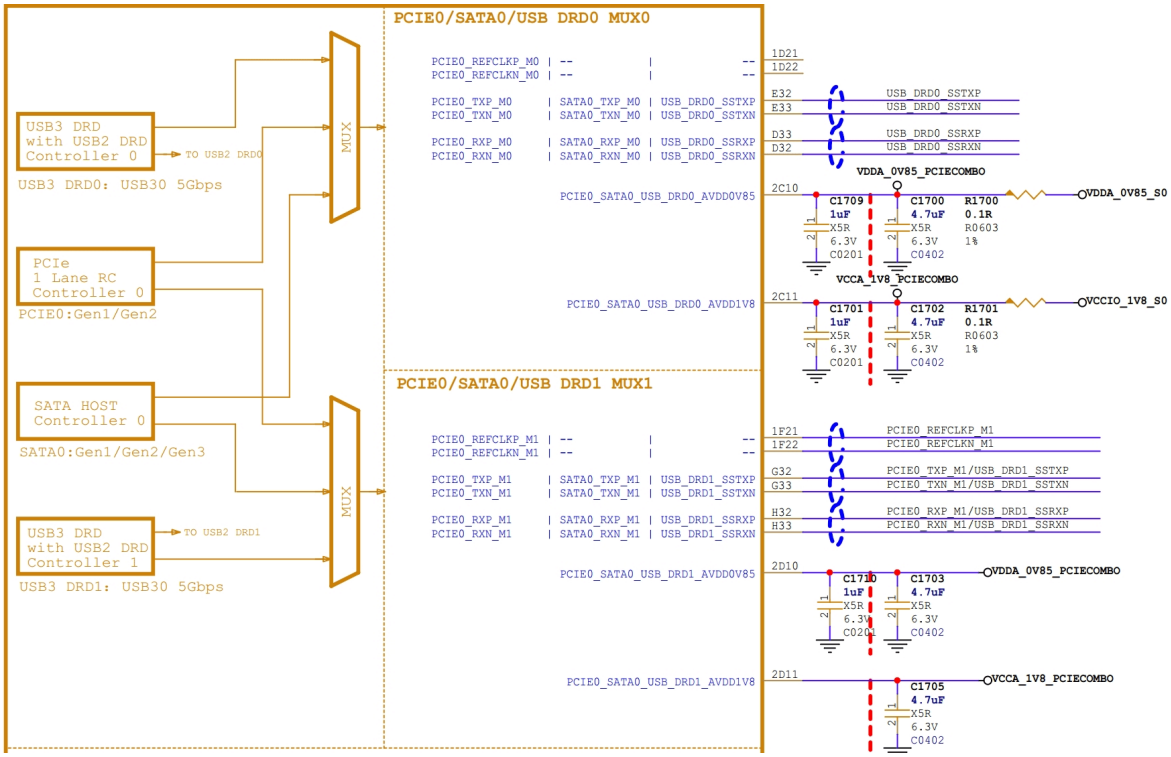


图 3 RK3572 USB3.0/SATA/PCIe Combo PHY0/1 供电电源

表 5 USB3.0/SATA/PCIe Combo PHY0/1 供电电压要求

PHY	供电电源	最小	正常	最大	Unit
Combo PHY0	PCIE0_SATA0_USB_DRD0_AVDD0V85	0.8	0.85	0.935	V
Combo PHY0	PCIE0_SATA0_USB_DRD0_AVDD1V8	1.62	1.8	1.98	V
Combo PHY1	PCIE0_SATA0_USB_DRD1_AVDD0V85	0.8	0.85	0.935	V
Combo PHY1	PCIE0_SATA0_USB_DRD1_AVDD1V8	1.62	1.8	1.98	V

USB3.0/SATA/PCIe Combo PHY 的功耗控制策略如下：

1. 芯片上电时，USB3.0/SATA/PCIe Combo PHY0/1 默认均处于工作状态。软件在 U-Boot SPL 阶段，设置 Combo PHY0/1 处于 reset state，以保持 PHY 处于最低功耗。进入内核后，USB 控制器驱动会通过调用 Combo PHY 的初始化函数释放 reset。
2. PHY 的动态功耗控制：当 Combo PHY 工作在 USB mode 时，PHY 的 PIPE state (P0/P1/P2/P3) 由 USB 控制器硬件自动控制，根据不同的工作场景，动态进入和退出 P0/P1/P2/P3 state。比如，未插入任何 USB 设备时，PIPE 处于 P3 state；插入 U3 disk 时，则切换到 P0 state；当接 U3 HUB 时，只要 HUB 的下行端口没有接其他 USB 外设，则 PIPE state 会自动进入 P3 state 低功耗。当有 USB 外设插入U3 HUB，则 PIPE state 自动切换为 P0。(注：P0 为正常工作状态，P3 为最低功耗状态)
3. 当明确不使用 USB3.0/SATA/PCIe Combo 接口时，对应的 Combo PHY 可以不供电，但 DTS 要修改配置，具体请参考 USB3.0 Combo PHY 不供电的 DTS 配置；
4. 在 Combo PHY 供电的情况下，如果不使用这个 PHY，需要将对应的 PHY DTS 节点配置为 disabled，这样可以让 PHY 处于 reset state，功耗最低。

表 6 USB 3.0/SATA/PCIe Combo PHY 功耗数据

供电电源	读写数据	动态休眠	PHY disabled	二级待机	Unit
AVDD_0V85	44.5	9.6	0.4	0	mA
AVDD_1V8	5.2	0.5	0.2	0	mA

Note:

1. 读写数据功耗的测试场景：接 U3 盘拷贝数据，PHY 处于 P0 state；
2. 动态休眠的测试场景：PHY DTS enable，但不接 USB 外设，PHY 处于 P3 State；
3. PHY disabled 的测试场景：PHY 的 DTS 节点配置为 disabled，PHY 处于 reset state；
4. 二级待机功耗的测试场景：PHY 的两路供电电源全部关闭。

2.3 USB 硬件电路设计注意点

2.3.1 USB_DRD0_VBUSDET 电路设计

USB_DRD0_VBUSDET 用于 USB 作为 Device 时，检测设备的连接和断开状态。其设计注意点如下：

1. 设计为支持 PD 功能的 Type-C 接口，即支持外置 Type-C 控制器芯片（FUSB302 或 HUSB311 等），则参考 RK3572 EVB1 Type-C 的电路设计（USB_DRD0_VBUSDET 固定上拉到 VCC_3V3_S0），软件驱动可以通过 Type-C 控制器芯片的 CC 检测 USB Device 的连接和断开;
2. 对于其他没有支持外置 Type-C 控制器芯片的电路设计方案（如Type-C USB 2.0 only，Type-A USB 3.0，Micro USB 2.0/3.0），要求 USB_DRD0_VBUSDET 仍然按照传统的分压电路设计，连接到

USB 接口的 VBUS 引脚，VBUSDET 不作常供电的设计（如果 USB_DRD0 有作 USB HOST 的需求，需要独立的 GPIO 或者 PMIC VBUS 控制，不与USB_DRD1 HOST VBUS 的控制电路复用）；

3. 要求芯片输入端 USB_DRD0_VBUSDET 的高电平范围在 **[0.9V ~ 3.3V]**

2.3.2 Maskrom USB 电路设计

Maskrom 固定使用 USB DRD0 接口下载固件，最高支持 USB3.0 下载固件，同时向下兼容 USB2.0 下载固件。硬件设计要求如下：

1. USB 2.0 PHY0 三路供电电源 USB_DRD_DVDD0V75/USB_DRD_AVDD1V8/USB_DRD_AVDD3V3 必须供电，具体请参考USB 2.0 PHY 供电及功耗管理；
2. USB3.0/SATA/PCIe Combo PHY0 相关供电电源，可以根据实际项目需求选择正常供电或者断电两种电路设计，具体请参考USB 3.0 PHY 供电及功耗管理；
3. 如果要支持 USB3.0 下载固件，要求 Combo PHY0 相关电源必须供电，并且 USB_DRD0_SSTXP/USB_DRD0_SSTXN, USB_DRD0_SSRXP/USB_DRD0_SSRXN 连接到 USB 下载口。
4. Maskrom USB 枚举不依赖于 USB_DRD0_VBUSDET 的电平；
5. 烧写工具配置和 U-Boot 配置说明，请参考RK3572 USB 下载固件的功能配置。

2.3.3 Type-C USB 3.0 硬件电路

USB3.0/SATA/PCIe Combo PHY 只支持 1 组 Tx/Rx，如果要支持 Type-C USB 3.0 的正反面硬件电路设计，需要增加 USB Switch 外围芯片，RK3572 SDK 提供两种 Type-C USB 3.0 硬件电路参考设计。

1. Type-C USB 3.0 Switch 硬件电路设计

   方案优点：软硬件成熟度高，只需要占用一个 USB DRD0 接口，软件控制简单；

   方案缺点：USB 3.0 Switch 的芯片成本较高，且对 USB 3.0 Switch 物料有品质要求，如果选用的物料较差，可能会影响 USB 3.0 信号质量。

2. Type-C USB 2.0 Switch 硬件电路设计

   方案优点：USB 2.0 Switch 芯片成本较低，且 USB3.0 信号质量不受 Switch 芯片的影响；

   方案缺点：需要同时占用 USB DRD0/1 两个控制器，没有额外的 USB 控制器作 Host 使用。同时，软件控制比较复杂，需要 OS 和内核驱动联动。

2.3.3.1 Type-C USB 3.0 Switch 硬件电路设计

以 RK3572 EVB1 Type-C USB 3.0 硬件电路设计为例，电路框图如下图所示，本章节主要说明设计注意点，详细的原理图设计请参考 SDK EVB1 原理图。

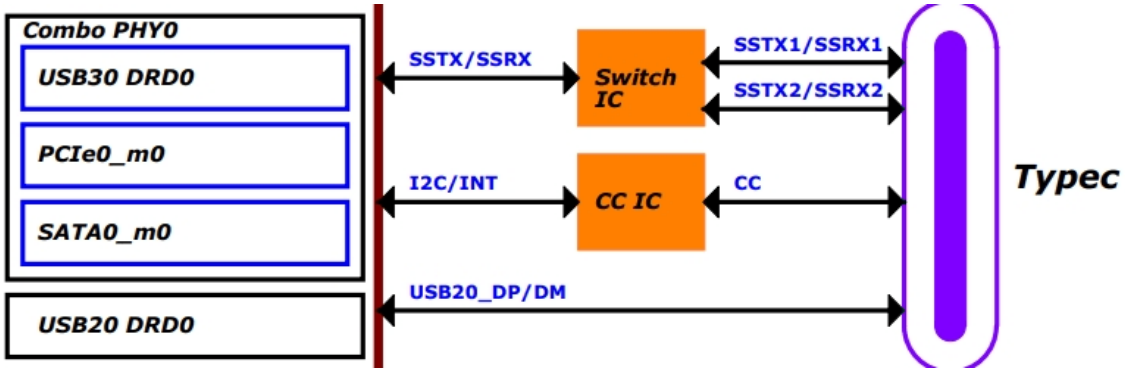


图 4 RK3572 Type-C USB 3.0 Switch 硬件电路设计框图

Type-C USB 3.0 Switch 硬件电路设计注意点如下：

1. 图 4 中的 CC IC 是外置的 Type-C 控制器芯片，用于实现 Type-C 正反面检测和 PD 充电协商。RK3572 可支持多个不同型号的 Type-C 控制器芯片，具体请参考Type-C 控制器芯片支持列表；
2. 图 4 中的 Switch IC 是外置的 USB 3.0 Switch 芯片，用于实现 Type-C 正反面 SSTX1/SSRX1 和 SSTX2/SSRX2 切换。同时，USB 2.0 DP/DM 可以直接通过 PCB 走线一分为二连接到 Type-C 接口的正反面；
3. 为了支持高压充电功能，同时降低芯片引脚的过压风险，USB_DRD0_VBUSDET 不要直连到 Type-C 接口的 VBUS，固定上拉到 3.3V 即可，但不能悬空；
4. USB_DRD0_ID 只用于 Micro 接口类型的 OTG 功能，Type-C 电路不需要使用，悬空即可；
5. 对应的 USB DTS 软件配置，请参考Type-C USB 3.0 Switch 全功能 DTS 配置。

2.3.3.2 Type-C USB 2.0 Switch 硬件电路设计

以 RK3572 EVB2 Type-C USB 3.0 硬件电路设计为例，电路框图如下所示，本章节主要说明设计注意点，详细的原理图设计请参考 SDK EVB2 原理图。

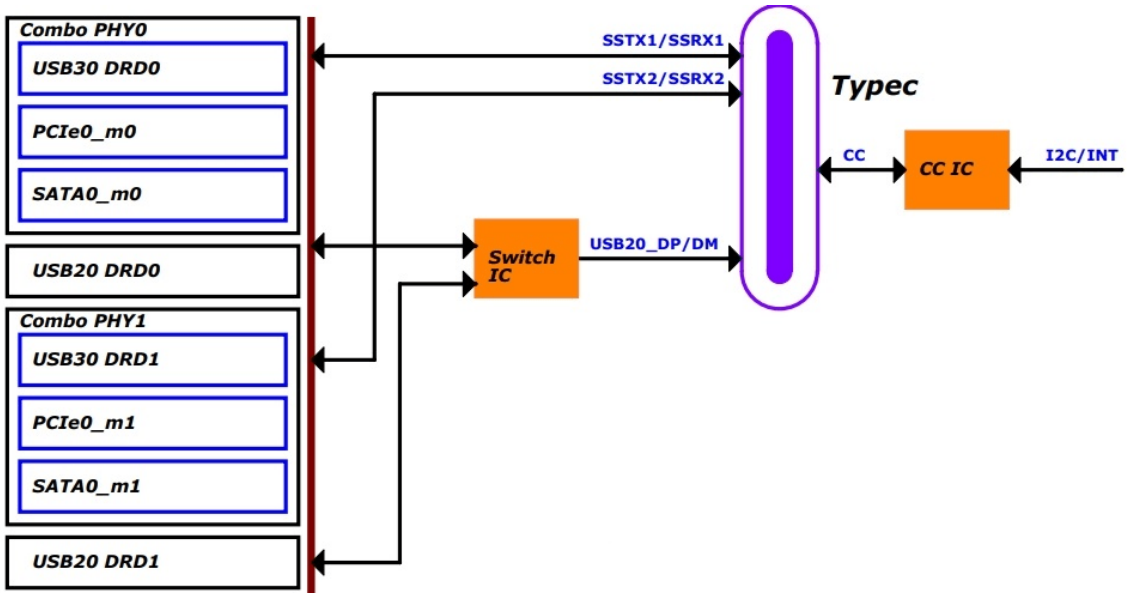


图 5 RK3572 Type-C USB 2.0 Switch 硬件电路设计框图

Type-C USB 2.0 Switch 的拓扑结构如下所示。

```
                          ------------------------
                          |    USB-C Connector   |
                          ------------------------
                              /              \
          ------------------------        ------------------------
          |  Orientation Normal  |        | Orientation Reverse  |
          ------------------------        ------------------------
               |              \_______________/              |
-----------------------              |              -----------------------
|   USB DRD0 Tx/Rx    |     -------------------     |   USB DRD1 Tx/Rx    |
-----------------------    /   USB20 SWITCH    \    -----------------------
                           --------------------
                             /              \
                   -------------------    -------------------
                   | USB DRD0 DP/DM |     | USB DRD1 DP/DM |
                   -------------------    -------------------
```

Type-C USB 2.0 Switch 硬件电路设计注意点如下：

1. 图 5 中 Type-C 接口的 SSTX1/SSRX1 连接到 USB DRD0 控制器的 SSTX/SSRX，而 Type-C 接口的 SSTX2/SSRX2 连接到 USB DRD1 控制器的 SSTX/SSRX。需要注意的是，不支持硬件设计直接将同一个 USB DRD 控制器的 SSTX/SSRX 通过 PCB 走线一分为二连接到 Type-C 的正反面，这样会影响 USB 5G 信号传输；
2. 图 5 中 的 CC IC 是外置的 Type-C 控制器芯片，用于实现 Type-C 正反面检测和 PD 充电协商。RK3572 可支持多个不同型号的 Type-C 控制器芯片，具体请参考Type-C 控制器芯片支持列表；
3. 图 5 中的 Switch IC 是外置的 USB2.0 Switch，用于实现 Type-C 正反面 USB DRD0 DP/DM 和 USB DRD1 DP/DM 切换。这样设计的目的是，保证 Type-C 正反面的 USB 3.0 和 USB 2.0 接口同属于一个 USB DRD 控制器 （如：Type-C 正面 SSTX1/SSRX1 和 DP/DM 同属于 USB DRD0），可由 USB 控制器硬件自动执行 USB 3.0 和 USB 2.0 模式切换；
4. 为了支持高压充电功能，同时降低芯片引脚的过压风险，USB_DRD0_VBUSDET 不要直连到 Type-C 接口的 VBUS，固定上拉到 3.3V 即可，但不能悬空；
5. USB_DRD0_ID 只用于 Micro 接口类型的 OTG 功能，Type-C 电路不需要使用，悬空即可；
6. 对应的 USB DTS 软件配置，请参考Type-C USB 2.0 Switch 全功能 DTS 配置。

2.3.4 Type-C USB 2.0 with PD 硬件电路

以 RK3572 Tablet DEMO1 Type-C USB 2.0 带 PD 硬件电路设计为例，电路框图如下所示，本章节主要说明设计注意点，详细的原理图设计请参考 SDK DEMO1 原理图。

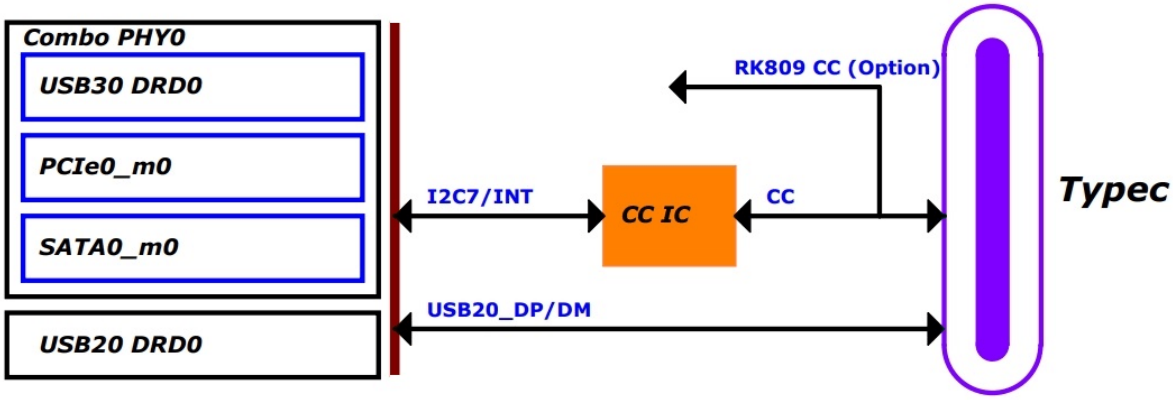


图 6 RK3572 Type-C USB 2.0 带 PD 硬件电路设计框图

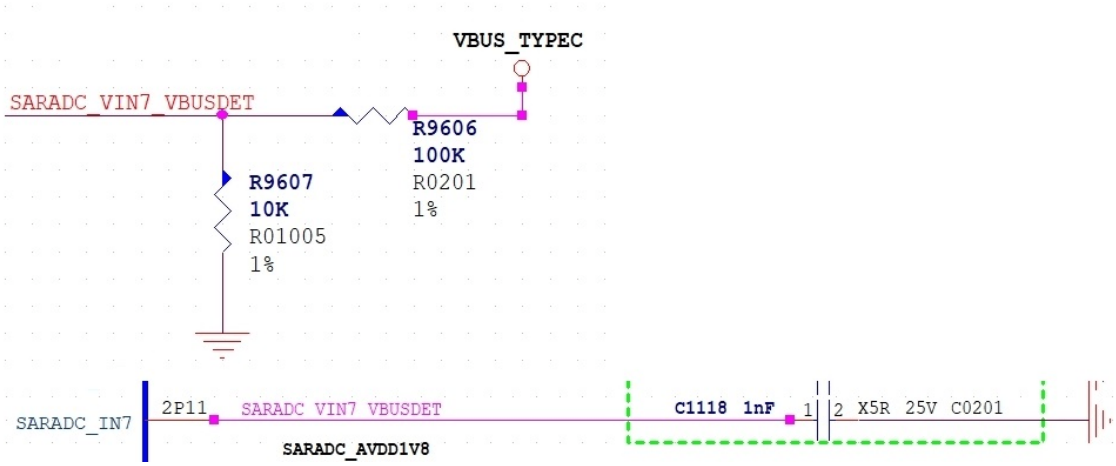


图 7 RK3572 SARADC VBUS 检测电路

Type-C USB 2.0 带 PD 的硬件电路设计注意点如下：

1. 图 6 中 的 CC IC 是外置的 Type-C 控制器芯片，用于实现 PD 充电协商。Tablet DEMO1 提供了两种 CC 硬件设计方案，除了常规的 CC IC 设计方案（HUSB311），还支持 PMIC RK809C PD 协

商。如果选用 RK809C 协商 PD，需要使用一路 SARADC 通过分压电路采集 Type-C 接口的 VBUS 电压，用于 Type-C 认证测试 TD.4.7.2 中的 VSAFE0V 检测，参考电路如图 7 所示；

2. USB 2.0 DP/DM 可以直接通过 PCB 走线一分为二连接到 Type-C 接口的正反面，不需要 USB2.0 Switch；
3. Combo PHY0 可以用于 PCIe0 或者 SATA0，如果 PCIe0 或者 SATA0 都没有使用，则 Combo PHY0 允许不供电，对应的供电电源引脚 PCIE0_SATA0_USB_DRD0_AVDD0V85/PCIE0_SATA0_USB_DRD0_AVDD1V8 接地；
4. 为了支持高压充电功能，同时降低芯片引脚的过压风险，USB_DRD0_VBUSDET 不要直连到 Type-C 接口的 VBUS，固定上拉到 3.3V 即可，但不能悬空；
5. USB_DRD0_ID 只用于 Micro 接口类型的 OTG 功能，Type-C 电路不需要使用，悬空即可；
6. 对应的 USB DTS 软件配置，请参考Type-C USB 2.0 with PD DTS 配置。

2.3.5 Type-C USB 2.0 only 硬件电路

以 RK3572 IOTEST Type-C USB 2.0 only 硬件电路设计为例，电路框图如下所示，本章节主要说明设计注意点，详细的原理图设计请参考 SDK IOTEST 原理图。

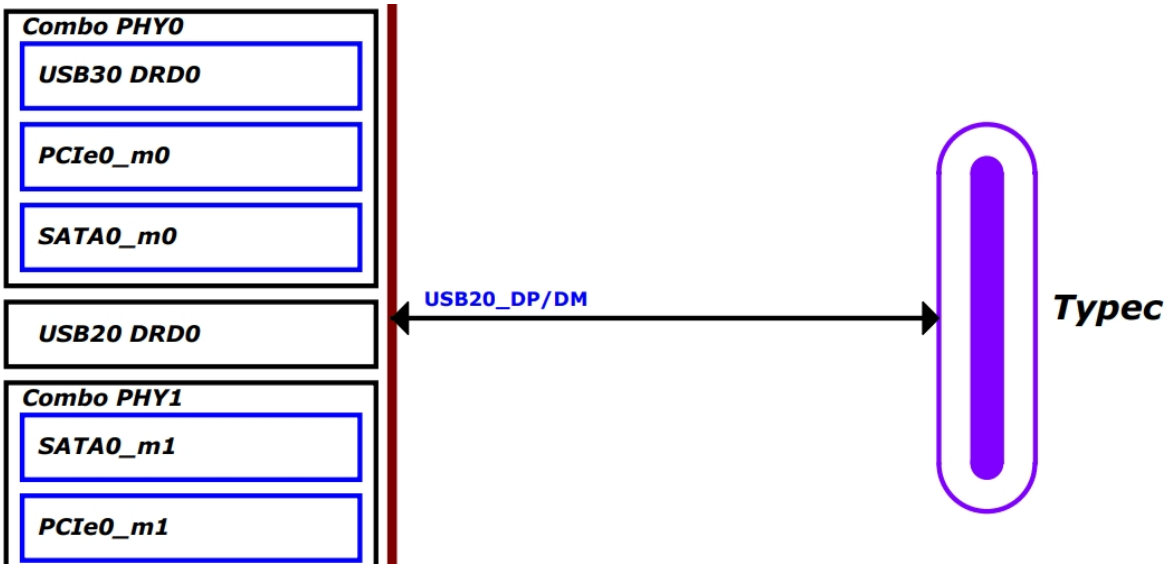


图 8 RK3572 Type-C USB 2.0 不带 PD 硬件电路设计框图

Type-C USB 2.0 不带 PD 的硬件电路设计注意点如下：

1. 该硬件方案不支持 PD 高压大电流协商，要求 USB_DRD0_VBUSDET 仍然按照传统的分压电路设计，连接到 Type-C 接口的 VBUS 引脚，不作常供电的设计，也不能悬空；
2. 该硬件方案不支持 USB DFP/UFP 自动切换，如果需要切换 DFP/UFP，建议使用RK3572 USB DRD mode 切换命令；
3. USB 2.0 DP/DM 可以直接通过 PCB 走线一分为二连接到 Type-C 接口的正反面，不需要 USB2.0 Switch；
4. Combo PHY0 可以用于 PCIe0 或者 SATA0，如果 PCIe0 或者 SATA0 都没有使用，则 Combo PHY0 允许不供电，对应的供电电源引脚 PCIE0_SATA0_USB_DRD0_AVDD0V85/PCIE0_SATA0_USB_DRD0_AVDD1V8 接地；
5. USB_DRD0_ID 只用于 Micro 接口类型的 OTG 功能，Type-C 电路不需要使用，悬空即可；
6. 如果需要兼容 USB-C to C 的线缆，且产品上只有 USB Device mode 的应用需求，建议将 Type-C 接口的 CC1/CC2 同时下拉 5.1KΩ 到地；
7. 对应的 USB DTS 软件配置，请参考Type-C USB 2.0 only DTS 配置。

2.3.6 Type-A USB 3.0 硬件电路

以 RK3572 EVB1 Type-A USB 3.0 硬件电路设计为例，电路框图如下图所示，本章节主要说明设计注意点，详细的原理图设计请参考 SDK EVB1 原理图。

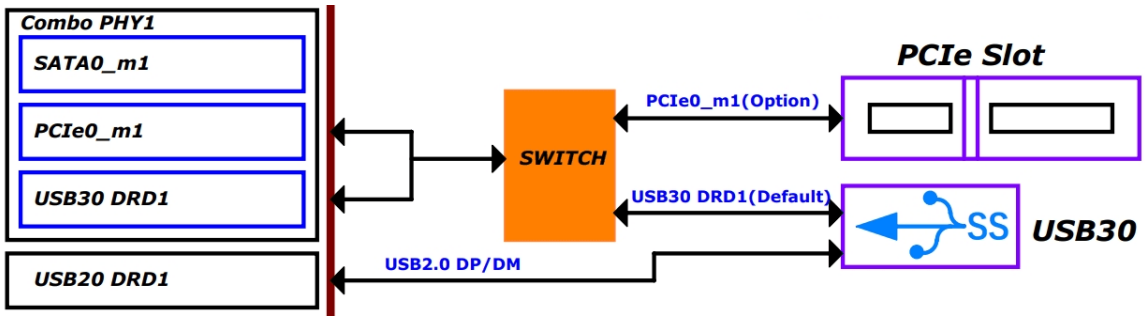


图 9 RK3572 Type-A USB 3.0 硬件电路设计框图

Type-A USB 3.0 硬件电路设计注意点如下：

1. 图 9 中的 SWITCH IC 用于切换 Combo PHY 的 USB 3.0 通路和 PCIe 通路。但 SDK 提供的软件驱动不支持动态切换 PCIe 和 USB 3.0 功能，如果硬件上切换到 PCIe 通路，需要重新配置 DTS 和编译内核，DTS 修改请参考 SDK 代码工程 `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10-pcie.dts`；
2. USB_DRD1_VBUSDET 用于 Device mode 的连接和断开检测。EVB1 Type-A USB 3.0 固定配置为 Host mode，不需要使用到 USB_DRD1_VBUSDET，可以外部上拉到 3.3V 或者悬空；
3. USB_DRD1_ID 只用于 Micro 接口类型的 OTG 功能，Type-A 电路不需要使用，悬空即可；
4. 对应的 USB DTS 软件配置，请参考Type-A USB 3.0 DTS 配置。

2.3.7 Multi Type-A USB 3.0 硬件电路

以 RK3572 EVB3 支持 2 x Type-A USB3.0 + 3 x Type-A USB2.0 硬件电路设计为例，电路框图如下图所示，本章节主要说明设计注意点，详细的原理图设计请参考 SDK EVB3 原理图。

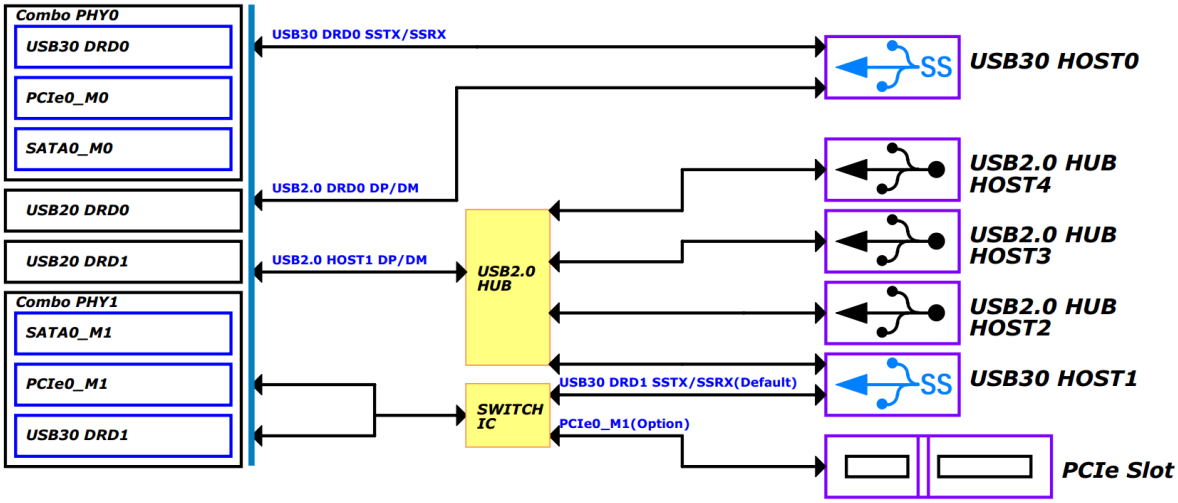


图 10 RK3572 Multi Type-A USB 3.0 硬件电路设计框图

Multi Type-A USB 3.0 硬件电路设计注意点如下：

1. RK3572 支持两个完整 USB 3.0 DRD 控制器，且作 USB Host 时，USB 2.0 Pin 和 USB 3.0 Pin 支持拆分独立同时使用，而作 USB Device 时，USB 2.0 Pin 和 USB 3.0 Pin 不支持独立同时使用。考虑到 USB DRD0 通常需要支持 Device mode（如：下载固件/ADB调试），因此，硬件设计上，要求 USB DRD0 的 USB 2.0 Pin 和 USB 3.0 Pin 连接到同一个 Type-A USB 3.0 接口上，不能拆分使用；
2. USB DRD1 固定作 Host mode 时，对应的 USB 2.0 Pin 和 USB 3.0 Pin 可以拆分使用。其中，USB 2.0 可以通过 USB 2.0 HUB 扩展出多个 USB 2.0 接口，并且，HUB 的一个下行 USB 2.0 端口可以和 USB DRD1 的 USB 3.0 Pin 组合为一个完整的 Type-A USB 3.0 接口；

3. USB_DRD0_VBUSDET 用于 Device mode 的连接和断开检测，要求按照传统的分压电路设计，连接到对应的 Type-A 接口 VBUS 引脚。USB_DRD1_VBUSDET 不需要支持 Device mode 的连接和断开检测，可以外部上拉到 3.3V 或者悬空；
4. 图 10 中的 SWITCH IC 用于切换 Combo PHY 的 USB 3.0 通路和 PCIe 通路。但 SDK 提供的软件驱动不支持动态切换 PCIe 和 USB 3.0 功能，如果硬件上切换到 PCIe 通路，需要重新配置 DTS 和编译内核，DTS 修改请参考 SDK 代码工程 `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10-pcie.dts`；
5. USB_DRD0_ID/USB_DRD1_ID 只用于 Micro 接口类型的 OTG 功能，Type-A 电路不需要使用，悬空即可；
6. 对应的 USB DTS 软件配置，请参考Multi Type-A USB 3.0 DTS 配置。

3. RK3572 USB DTS 配置

RK3572 USB DTS 配置，包括：芯片级 USB 控制器/PHY DTSI 配置和板级 DTS 配置。详细配置方法，请参考内核如下文档：

1. kernel/Documentation/devicetree/bindings/usb/snps,dwc3.yaml (USB Controller)
2. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-inno-usb2.yaml (USB 2.0 PHY)
3. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-naneng-combphy.yaml (USB 3.0 Combo PHY)
4. kernel/Documentation/devicetree/bindings/connector/usb-connector.yaml (Type-C connector)
5. kernel/Documentation/devicetree/bindings/usb/hynetek,husb311.yaml (HUSB311 TCPC Chip)
6. kernel/Documentation/devicetree/bindings/usb/rockchip,pmic-tcpc.yaml (RK809C TCPC Chip)

3.1 USB 芯片级 DTSI 配置

RK3572 DTSI 文件中 USB 控制器和 PHY 相关的主要节点，如下表所示，由于 USB DTSI 节点配置的是 USB 控制器和 PHY 的公共资源和属性，建议开发者不要改动。对应的 DTSI 完整路径如下：

`arch/arm64/boot/dts/rockchip/rk3572.dtsi`

表 7 RK3572 USB 接口和 USB DTS 节点的对应关系

USB 接口名称(原理图)	USB 控制器 DTS 节点	USB PHY DTS 节点
USB DRD0	usb_drd0_dwc3	u2phy0 u2phy0_otg combphy0_psu
USB DRD1	usb_drd1_dwc3	u2phy1 u2phy1_otg combphy1_psu

3.1.1 USB 控制器 DTSI 配置

```
/* USB DRD0 Controller (USB2.0/3.0) */
usb_drd0_dwc3: usb@24000000 {
        compatible = "rockchip,rk3572-dwc3", "rockchip,rk3576-dwc3",
"snps,dwc3";
        ......
};

/* USB DRD1 Controller (USB2.0/3.0) */
usb_drd1_dwc3: usb@24400000 {
        compatible = "rockchip,rk3572-dwc3", "rockchip,rk3576-dwc3",
"snps,dwc3";
        ......
};
```

USB 控制器默认支持 LPM 功能和动态 suspend USB2/USB3 PHY 功能，可以降低 USB 模块运行功耗。同时，支持通过 DTS 属性配置关闭对应的功能。详细的 DTS 配置说明，请参考章节 USB 控制器 DTS 属性配置说明。

3.1.2 USB PHY DTSI 配置

USB 2.0/3.0 PHY 和 USB 控制器具有一对一的映射关系，需要成对配置，且不支持交换 PHY 和 控制器映射。在芯片内部，USB PHY 和 控制器的连接关系，请参考 RK3572 USB 控制器和 PHY 简介。在 DTSI 节点中，USB 控制器通过节点的 "phys" 属性关联对应的 USB PHY。

```
/* USB 2.0 PHY0/1 */
usb2phy_grf: syscon@26054000 {
        compatible = "rockchip,rk3572-usb2phy-grf", "syscon", "simple-mfd";
        reg = <0x0 0x26054000 0x0 0x4000>;
        ......
        /* PHY0对应USB DRD0控制器 */
        u2phy0: usb2-phy@0 {
                compatible = "rockchip,rk3572-usb2phy";
                reg = <0x0 0x10>;
                ......
                u2phy0_otg: otg-port {
                        ......
                };
        };
        /* PHY1对应USB DRD1控制器 */
        u2phy1: usb2-phy@2000 {
                compatible = "rockchip,rk3572-usb2phy";
                reg = <0x2000 0x10>;
                ......
                u2phy1_otg: otg-port {
                        ......
                };
        };
};

/* USB3.0/SATA/PCIe Combo PHY0,对应USB DRD0控制器 */
combphy0_psu: phy@2c410000 {
        compatible = "rockchip,rk3572-pipe-combphy";
        reg = <0x0 0x2c410000 0x0 0x100>;
        ......
```

```
};

/* USB3.0/SATA/PCIe Combo PHY1，对应USB DRD1控制器 */
combphy1_psu: phy@2c420000 {
        compatible = "rockchip,rk3572-pipe-combphy";
        reg = <0x0 0x2c420000 0x0 0x100>;
        ......
};
```

USB 2.0 PHY 作 Device mode 时，默认支持充电检测和动态进入 suspend 功能。同时，支持通过 DTS 属性配置关闭对应的功能。详细的 USB 2.0 PHY 属性配置说明，请参考章节 USB 2.0 PHY DTS 属性配置说明。

USB 3.0/SATA/PCIe Combo PHY0/1 的属性配置说明，请参考章节 USB 3.0 Combo PHY DTS 属性配置说明。

3.2 Type-C USB 3.0 Switch 全功能 DTS 配置

参考 `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10.dtsi` Type-C USB 3.0 接口的 DTS 配置。

```
/* 配置USB3.0 Switch，"select-gpios"对应硬件SEL Pin，用于支持Type-C正反面Tx/Rx切换 */
usb3_switch: usb3-switch {
        compatible = "broadchip,bct4360", "gpio-sbu-mux";
        select-gpios = <&gpio1 RK_PD4 GPIO_ACTIVE_HIGH>;
        pinctrl-names = "default";
        pinctrl-0 = <&usb3_switch_default>;
        orientation-switch;
};

/* 使能USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        status = "okay";
};

/* USB2.0 PHY父节点配置属性"orientation-switch"，用于注册Type-C orientation switch回调函数和控制Bvalid信号 */
&u2phy0 {
        orientation-switch;
        status = "okay";
};

/* USB2.0 PHY子节点配置属性"rockchip,typec-vbus-det"，表示支持Type-C VBUSDET引脚常拉高的硬件设计 */
&u2phy0_otg {
        rockchip,typec-vbus-det;
        status = "okay";
};

/* 配置USB3.0 DRD0 Controller */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        usb-role-switch; /* 使能注册Type-C USB Role Switch回调函数用于切换DFP/UFP */
        status = "okay";
};
```

```
/* 配置外置Type-C控制器芯片HUSB311 */
&i2c1 { /* 实际的硬件电路设计HUSB311不限制挂在I2C1，需要根据实际挂的I2C总线进行配置 */
        status = "okay";

        usbc0: husb311@4e {
                compatible = "hynetek,husb311";
                reg = <0x4e>; /* 配置HUSB311的I2C地址 */
                interrupt-parent = <&gpio0>; /* 配置HUSB311的中断脚对应的GPIO */
                interrupts = <RK_PA5 IRQ_TYPE_LEVEL_LOW>; /* 配置低电平产生中断 */
                pinctrl-names = "default";
                pinctrl-0 = <&usbc0_int>;
                vbus-supply = <&vbus5v0_typec>; /* 配置Type-C VBUS属性，用于控制输出
5V给外设 */
                status = "okay";

                /* connector详细配置，参考内核文档 usb-connector.yaml */
                usb_con0: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual"; /* 支持host/device两种data角色 */
                        power-role = "dual"; /* 支持source/sink两种power角色 */
                        try-power-role = "sink"; /* 优先作为sink角色 */
                        op-sink-microwatt = <1000000>; /* 接收端所需的运行功率，单位
是微瓦 */
                        orientation-switch = <&usb3_switch &u2phy0>; /* 配置orien
switch linked节点 */
                        usb-role-switch = <&usb_drd0_dwc3>; /* 配置role switch
linked节点 */

                        /* 配置实际可支持的sink和source能力 */
                        sink-pdos = <PDO_FIXED(5000, 1000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EX
TPOWER | PDO_FIXED_DUAL_ROLE)>;
                        source-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP
| PDO_FIXED_USB_COMM | PDO_FIXED_
EXTPOWER | PDO_FIXED_DUAL_ROLE)>;
                };
        };
};
```

Note：使用 FUSB302/AW35615 芯片替代 HUSB311 芯片的软件修改

1. DTS 修改：只需要基于 HUSB311 的 DTS 配置进行简单修改即可，参考修改如下：

```
/* 配置外置Type-C控制器芯片FUSB302 */
&i2c2 {
        usbc0: fusb302@22 {
                compatible = "fcs,fusb302";
                reg = <0x22>; /* 配置FUSB302的I2C地址 */
                ..... /* 其它配置同HUSB311 */
        };
};
```

2. 驱动软件修改：修改 FUSB302 驱动的 irq wakeup 机制，以支持待机 VBUS 断电或者供电等各种应用场景。

```
diff --git a/drivers/usb/typec/tcpm/fusb302.c b/drivers/usb/typec/tcpm/fusb302.c
index 77a050cd2e88..72bef94887b4 100644
--- a/drivers/usb/typec/tcpm/fusb302.c
+++ b/drivers/usb/typec/tcpm/fusb302.c
@@ -105,6 +105,7 @@ struct fusb302_chip {
        bool vbus_on;
        bool charge_on;
        bool vbus_present;
+       bool wakeup;
        enum typec_cc_polarity cc_polarity;
        enum typec_cc_status cc1;
        enum typec_cc_status cc2;
@@ -1769,7 +1770,8 @@ static int fusb302_probe(struct i2c_client *client,
                dev_err(dev, "cannot request IRQ for GPIO Int_N, ret=%d", ret);
                goto tcpm_unregister_port;
        }
-       enable_irq_wake(chip->gpio_int_n_irq);
+       chip->wakeup = device_property_read_bool(dev, "wakeup-source");
+       device_init_wakeup(dev, true);
        i2c_set_clientdata(client, chip);

        return ret;
@@ -1788,7 +1790,7 @@ static void fusb302_remove(struct i2c_client *client)
        struct fusb302_chip *chip = i2c_get_clientdata(client);

-       disable_irq_wake(chip->gpio_int_n_irq);
+       device_init_wakeup(chip->dev, false);
        free_irq(chip->gpio_int_n_irq, chip);
        kthread_destroy_worker(chip->irq_worker);
        cancel_delayed_work_sync(&chip->bc_lvl_handler);
@@ -1809,6 +1811,11 @@ static int fusb302_pm_suspend(struct device *dev)

        /* Make sure any pending irq_work is finished before the bus suspends */
        kthread_flush_worker(chip->irq_worker);
+
+       if (device_may_wakeup(dev) && (!chip->vbus_on || chip->wakeup))
+               enable_irq_wake(chip->gpio_int_n_irq);
+       else
+               disable_irq(chip->gpio_int_n_irq);
        return 0;
 }

@@ -1819,6 +1826,11 @@ static int fusb302_pm_resume(struct device *dev)
        u8 pwr;
        int ret = 0;

+       if (device_may_wakeup(dev) && (!chip->vbus_on || chip->wakeup))
+               disable_irq_wake(chip->gpio_int_n_irq);
+       else
+               enable_irq(chip->gpio_int_n_irq);
+
        /*
         * When the power of fusb302 is lost or i2c read failed in PM S/R
         * process, we must reset the tcpm port first to ensure the devices
```

3.3 Type-C USB 2.0 Switch 全功能 DTS 配置

参考 `arch/arm64/boot/dts/rockchip/rk3572-evb2-v10.dtsi` Type-C USB 3.0 接口的 DTS 配置。

```
/* 配置extcon usb驱动，用于接收TCPM orien/role通知，并发送usb role extcon给对应的USB DRD0/1控制器驱动 */
extcon_usb: extcon-usb {
        compatible = "rockchip,extcon-usb";
        orientation-switch;
        usb-role-switch;

        extcon_usb_port0: extcon-usb-port0 {
                rockchip,usbc-orientation = <1>; /* 配置 extcon port0 对应 normal orien */
                extcon = <&u2phy0>; /* 固定引用u2phy0创建的extcon dev，用于通知u2phy当前的usb role */
        };

        extcon_usb_port1: extcon-usb-port1 {
                rockchip,usbc-orientation = <2>; /* 配置 extcon port1 对应 reverse orien */
                extcon = <&u2phy0>; /* 固定引用u2phy0创建的extcon dev，用于通知u2phy当前的usb role，注意不能引用u2phy1 */
        };
};

/* 配置USB2.0 Switch，"select-gpios"对应硬件SEL Pin，用于支持Type-C正反面DP/DM切换 */
usb2_switch: usb2-switch {
        compatible = "broadchip,bct4010", "gpio-sbu-mux";
        select-gpios = <&gpio1 RK_PD4 GPIO_ACTIVE_HIGH>; /* GPIO可以选择其它引脚，但要求上电默认为下拉，即默认选通DRD0 */
        pinctrl-names = "default";
        pinctrl-0 = <&usb2_switch_default>;
        orientation-switch;
};

/* 使能USB3.0/SATA/PCIe Combo PHY0，用于Type-C正面TX1/RX1 */
&combphy0_psu {
        status = "okay";
};
/* 使能USB3.0/SATA/PCIe Combo PHY1，用于Type-C反面TX2/RX2 */
&combphy1_psu {
        status = "okay";
};

/* USB2.0 PHY0父节点配置属性"orientation-switch"，用于注册Type-C orientation switch回调函数和控制Bvalid信号 */
&u2phy0 {
        orientation-switch;
        status = "okay";
};

/* USB2.0 PHY1父节点配置属性"rockchip,partner-phy"，用于表示和PHY0组合为Type-C partner port */
&u2phy1 {
        rockchip,partner-phy = <&u2phy0>;
```

```
        extcon = <&u2phy0>; /* 固定引用u2phy0创建的extcon dev，目的是统一USB BC1.2
extcon通知链 */
        status = "okay";
};

/* USB2.0 PHY0子节点配置属性"rockchip,typec-vbus-det"，表示支持Type-C VBUSDET引脚常拉
高的硬件设计 */
&u2phy0_otg {
        rockchip,typec-vbus-det;
        status = "okay";
};

/* USB2.0 PHY1子节点配置属性"rockchip,typec-vbus-det"，表示支持Type-C VBUSDET引脚常拉
高的硬件设计 */
&u2phy1_otg {
        rockchip,typec-vbus-det;
        status = "okay";
};

/* USB3.0 DRD0 Controller，属性"extcon"用于接收extcon usb驱动port0的通知切换DFP/UFP
*/
&usb_drd0_dwc3 {
        dr_mode = "otg";
        extcon = <&extcon_usb_port0>;
        status = "okay";
};

/* USB3.0 DRD1 Controller，属性"extcon"用于接收extcon usb驱动port1的通知切换DFP/UFP
*/
&usb_drd1_dwc3 {
        dr_mode = "otg";
        extcon = <&extcon_usb_port1>;
        status = "okay";
};

/* 配置外置Type-C控制器芯片HUSB311 */
&i2c1 { /* 实际的硬件电路设计HUSB311不限制挂在I2C1，需要根据实际挂的I2C总线进行配置 */
        status = "okay";

        usbc0: husb311@4e {
                compatible = "hynetek,husb311";
                reg = <0x4e>; /* 配置HUSB311的I2C地址 */
                interrupt-parent = <&gpio0>; /* 配置HUSB311的中断脚对应的GPIO */
                interrupts = <RK_PC1 IRQ_TYPE_LEVEL_LOW>; /* 配置低电平产生中断 */
                pinctrl-names = "default";
                pinctrl-0 = <&usbc0_int>;
                vbus-supply = <&vbus5v0_typec>; /* 配置Type-C VBUS属性，用于控制输出
5V给外设 */
                status = "okay";

                /* connector详细配置，参考内核文档 usb-connector.yaml */
                usb_con0: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual"; /* 支持host/device两种data角色 */
                        power-role = "dual"; /* 支持source/sink两种power角色 */
                        try-power-role = "sink";  /* 优先作为sink角色 */
```

```
                        op-sink-microwatt = <1000000>; /* 接收端所需的运行功率，单位
是微瓦 */
                        orientation-switch = <&extcon_usb &usb2_switch &u2phy0>;
/* 配置orien switch linked节点，最多支持3个 */
                        usb-role-switch = <&extcon_usb>; /* 配置role switch
linked节点 */

                        /* 配置实际可支持的sink和source能力 */
                        sink-pdos = <PDO_FIXED(5000, 1000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EX
TPOWER | PDO_FIXED_DUAL_ROLE)>;
                        source-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP
| PDO_FIXED_USB_COMM | PDO_FIXED_
EXTPOWER | PDO_FIXED_DUAL_ROLE)>;
                };
        };
};
```

3.4 Type-C USB 2.0 with PD DTS 配置

参考 `arch/arm64/boot/dts/rockchip/rk3572-tablet-v10.dts` Type-C USB 2.0 接口的 DTS 配置。

```
/* 配置 USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        rockchip,dis-u3otg0-port; /* 如果SATA/PCIe有使用Combo PHY0，需要加上该属性 */
        status = "okay"; /* 如果SATA/PCIe未使用Combo PHY0，这里改为status =
"disabled" */
};

/* 配置外置Type-C控制器芯片RK809C */
&i2c0 { /* 实际的硬件电路设计RK809C不限制挂在I2C0，需要根据实际挂的I2C总线进行配置 */
        status = "okay";

        usbc0: rk809c-tcpc@22 {
                compatible = "rockchip,rk809c-tcpc", "rockchip,rk817b2-tcpc";
                reg = <0x22>; /* 配置RK809C的I2C地址 */
                interrupt-parent = <&gpio0>; /* 配置RK809C的中断脚对应的GPIO */
                interrupts = <RK_PA6 IRQ_TYPE_LEVEL_LOW>; /* 配置低电平产生中断 */
                pinctrl-names = "default";
                pinctrl-0 = <&pmic_pins>;
                vbus-supply = <&vbus5v0_typec>; /* 配置Type-C VBUS属性，用于控制输出
5V给外设 */
                io-channels = <&saradc 7>; /* 配置Type-C VBUS对应的SARADC */
                io-channel-names = "vsafe0v-detect";
                /* rockchip,vsafe0v-det; 使能vsafe0v检测，可选属性，用于PD认证测试中
vsafe0v相关项的测试 */
                status = "okay";

                /* connector详细配置，参考内核文档 usb-connector.yaml */
                usb_con: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual"; /* 支持host/device两种data角色 */
                        power-role = "dual"; /* 支持source/sink两种power角色 */
                        try-power-role = "sink"; /* 优先作为sink角色 */
```

```
                        op-sink-microwatt = <1000000>; /* 接收端所需的运行功率，单位
是微瓦 */
                        orientation-switch = <&u2phy0>; /* 配置orien switch
linked节点，最多支持3个 */
                        usb-role-switch = <&usb_drd0_dwc3>;  /* 配置role switch
linked节点 */

                        /* 配置实际可支持的sink和source能力 */
                        sink-pdos =
                                <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOW
ER | PDO_FIXED_DUAL_ROLE)
                                 PDO_FIXED(9000, 3000, PDO_FIXED_USB_COMM)
                                 /*PDO_FIXED(12000, 3000,
PDO_FIXED_USB_COMM)*/>;
                        source-pdos =
                                <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOW
ER | PDO_FIXED_DUAL_ROLE)>;
                };
        };
};

/* USB2.0 PHY0父节点配置属性"orientation-switch"，用于注册Type-C orientation switch回
调函数和控制Bvalid信号 */
&u2phy0 {
        orientation-switch;
        status = "okay";
};

/* USB2.0 PHY0子节点配置属性"rockchip,typec-vbus-det"，表示支持Type-C VBUSDET引脚常拉
高的硬件设计 */
&u2phy0_otg {
        rockchip,typec-vbus-det;
        /* rockchip,sel-pipe-phystatus; 可选属性，选择GRF控制pipe phystatus，替代
USB3.0 ComboPHY的控制 */
        status = "okay";
};

/* 配置USB3.0 DRD0 Controller支持USB2.0 only模式 */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        maximum-speed = "high-speed"; /* 配置最高只支持HS */
        phys = <&u2phy0_otg>; /* 配置phys属性，只引用USB2.0 PHY0 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* 关闭suspend USB2.0 PHY的功能 */
        snps,usb2-lpm-disable; /* 关闭Host mode USB2 LPM功能 */
        snps,usb2-gadget-lpm-disable; /* 关闭Device mode USB2 LPM功能 */
        usb-role-switch; /* 使能注册Type-C USB Role Switch回调函数用于切换DFP/UFP */
        status = "okay";
};
```

3.5 Type-C USB 2.0 only DTS 配置

参考 `arch/arm64/boot/dts/rockchip/rk3572-iotest-v10.dtsi` Type-C USB 2.0 接口的 DTS 配置。

```
/* 配置USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        rockchip,dis-u3otg0-port; /* 如果SATA/PCIe有使用Combo PHY0，需要加上该属性 */
        status = "okay"; /* 如果SATA/PCIe未使用Combo PHY0，这里改为status =
"disabled" */
};

/* 使能USB2.0 PHY0父节点，因为不支持TCPC芯片，不需要额外的配置属性 */
&u2phy0 {
        status = "okay";
};

/* 使能USB2.0 PHY0子节点 */
&u2phy0_otg {
        rockchip,vbus-always-on; /* 可选属性，用于关闭USB2.0 PHY充电检测功能 */
        rockchip,sel-pipe-phystatus;  /* 选择GRF控制pipe phystatus，替代USB3.0
ComboPHY的控制 */
        status = "okay";
};

/* 配置USB3.0 DRD0 Controller支持USB2.0 only模式 */
&usb_drd0_dwc3 {
        dr_mode = "peripheral"; /* 配置Device only模式，不支持Host模式 */
        /* extcon = <&u2phy0>; 可选属性，用于dr_mode = "otg" */
        maximum-speed = "high-speed";  /* 配置最高只支持HS */
        phys = <&u2phy0_otg>; /* 配置phys属性，只引用USB2.0 PHY0 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* 关闭suspend USB2.0 PHY的功能 */
        snps,usb2-gadget-lpm-disable; /* 关闭Device mode USB2 LPM功能 */
        snps,usb2-lpm-disable; /* 关闭Host mode USB2 LPM功能 */
        status = "okay";
};
```

3.6 Type-A USB 3.0 DTS 配置

参考 `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10.dtsi` Type-A USB 3.0 接口的 DTS 配置。

```
/* 使能 USB3.0/SATA/PCIe Combo PHY1 */
&combphy1_psu {
        status = "okay";
};

/* 使能USB2.0 PHY1父节点 */
&u2phy1 {
        status = "okay";
};

/* 使能USB2.0 PHY1子节点 */
&u2phy1_otg {
        phy-supply = <&vcc5v0_usb3_host>; /* 配置VBUS供电控制的属性，注意不是PHY的供电
*/
```

```
        status = "okay";
};

/* 配置USB3.0 DRD1 Controller */
&usb_drd1_dwc3 {
        dr_mode = "host"; /* 配置Host only模式，不支持Device模式 */
        status = "okay";
};
```

3.7 Multi Type-A USB 3.0 DTS 配置

参考 `arch/arm64/boot/dts/rockchip/rk3572-evb3-v10.dtsi` Multi Type-A USB 3.0 接口（2 x Type-A USB3.0 + 3 x Type-A USB2.0）的 DTS 配置。

```
/* 配置外置USB2.0 HUB芯片的Reset Pin */
vcc3v3_hubreset: vcc3v3-hubreset {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_hubreset";
        ......
};

/* 配置USB3.0 DRD0接口的VBUS控制节点 */
vcc5v0_usb3_drd: vcc5v0-usb3-drd {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_usb3_drd";
        ......
};

/* 配置USB3.0 DRD1接口的VBUS控制节点 */
vcc5v0_usb3_host: vcc5v0-usb3-host {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_usb3_host";
        ......
};

/* 使能USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        status = "okay";
};

/* 使能USB3.0/SATA/PCIe Combo PHY1 */
&combphy1_psu {
        status = "okay";
};

/* 使能USB2.0 PHY0父节点 */
&u2phy0 {
        status = "okay";
};

/* 配置USB2.0 PHY0子节点 */
&u2phy0_otg {
        vbus-supply = <&vcc5v0_usb3_drd>; /* vbus-supply用于DRD mode，支持动态开关
VBUS */
        status = "okay";
```

```
};

/* 使能USB2.0 PHY1父节点 */
&u2phy1 {
        status = "okay";
};

/* 配置USB2.0 PHY1子节点 */
&u2phy1_otg {
        phy-supply = <&vcc5v0_usb3_host>; /* phy-supply用于HOST mode，系统上电后，
VBUS常开 */
        status = "okay";
};

/* 配置USB3.0 DRD0 Controller */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        extcon = <&u2phy0>; /* 配置extcon属性，用于接收USB2.0 PHY通知切换Device mode
*/
        status = "okay";
};

/* 配置USB3.0 DRD1 Controller */
&usb_drd1_dwc3 {
        dr_mode = "host"; /* 配置Host only模式，不支持Device模式 */
        status = "okay";
};
```

3.8 USB3.0 Combo PHY 不供电的 DTS 配置

USB3.0 Combo PHY0/1 对应的 USB 3.0/SATA0/PCIe0 接口在 未使用时，允许对应的外部供电电源不供电，可以降低系统功耗和简化硬件电路设计，但需要软件修改对应的 DTS 配置，以避免系统启动异常或者 USB 功能异常。

3.8.1 Combo PHY0 不供电的 DTS 配置

```
/* 关闭USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        status = "disabled";
};

/* 配置USB3.0 DRD0 Controller仅支持USB2.0 only模式 */
&usb_drd0_dwc3 {
        maximum-speed = "high-speed";  /* 配置最高只支持HS */
        phys = <&u2phy0_otg>; /* 配置phys属性，只引用USB2.0 PHY0 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* 关闭suspend USB2.0 PHY的功能 */
        snps,usb2-gadget-lpm-disable; /* 关闭Device mode USB2 LPM功能 */
        snps,usb2-lpm-disable; /* 关闭Host mode USB2 LPM功能 */
        status = "okay";
};
```

需要注意的是，烧写工具必须关闭 USB 3.0 下载功能，否则会出现 Maskrom 下载固件失败的问题，修改方法参考章节Rockusb2.0 下载工具配置。

3.8.2 Combo PHY1 不供电的 DTS 配置

```
/* 关闭USB3.0/SATA/PCIe Combo PHY1 */
&combphy1_psu {
        status = "disabled";
};

/* 配置USB3.0 DRD1 Controller仅支持USB2.0 only模式 */
&usb_drd1_dwc3 {
        maximum-speed = "high-speed";  /* 配置最高只支持HS */
        phys = <&u2phy1_otg>; /* 配置phys属性，只引用USB2.0 PHY1 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* 关闭suspend USB2.0 PHY的功能 */
        snps,usb2-gadget-lpm-disable; /* 关闭Device mode USB2 LPM功能 */
        snps,usb2-lpm-disable; /* 关闭Host mode USB2 LPM功能 */
        status = "okay";
};
```

3.9 USB DTS 重要属性配置说明

3.9.1 USB 控制器 DTS 属性配置说明

1. 属性 `usb-role-switch` 仅用于标准 Type-C 接口（带有 PD 控制器芯片），同时须配置 `dr_mode = "otg"` 属性。如果 `dr_mode` 为非 `"otg"` 模式，请勿配置；
2. 属性 `extcon` 主要功能之一是动态切换 DRD mode，适用于非标准 Type-C 接口的硬件电路设计，如：USB Micro 接口或者 Type-A 接口，同时控制器配置为 `dr_mode = "otg"` 模式的方案中，实现 Device/Host 模式动态切换；
3. 属性 `snps,usb2-lpm-disable` 用于关闭 DWC3 Host mode USB2 LPM 功能的属性配置

   ```
   &usb_drd0_dwc3 {
           snps,usb2-lpm-disable;
   };

   &usb_drd1_dwc3 {
           snps,usb2-lpm-disable;
   };
   ```

4. 属性 `snps,usb2-gadget-lpm-disable` 用于关闭 DWC3 Device mode USB2 LPM 功能的属性配置

   ```
   &usb_drd0_dwc3 {
           snps,usb2-gadget-lpm-disable;
   };
   ```

5. 属性 `snps,dis_u2_susphy_quirk` 和 `snps,dis_u3_susphy_quirk` 用于关闭 DWC3 suspend USB2.0/3.0 PHY 功能的属性配置

```
&usb_drd0_dwc3 {
        snps,dis_u2_susphy_quirk;
        snps,dis_u3_susphy_quirk;
};

&usb_drd1_dwc3 {
        snps,dis_u2_susphy_quirk;
        snps,dis_u3_susphy_quirk;
};
```

3.9.2 USB 2.0 PHY DTS 属性配置说明

1. 属性 `vbus-supply` 和属性 `phy-supply` 都是用于控制 VBUS 输出 5V，但两者又有使用区别。`vbus-supply` 用于 DRD mode，支持动态开关 VBUS。`phy-supply` 用于 USB HOST 口，系统上电后，保持 VBUS 5V 常开；
2. 属性 `rockchip,sel-pipe-phystatus` 用于配置 GRF USB 控制寄存器，选择 GRF 控制 pipe phystatus，替代 USB 3.0 Combo PHY 的控制。主要用于 USB 2.0 only 的方案；
3. 属性 `rockchip,typec-vbus-det` 用于支持 Type-C VBUS_DET 常拉高的硬件设计；
4. 属性 `rockchip,vbus-always-on` 用于关闭 USB 2.0 PHY 充电检测功能的属性配置

   ```
   &u2phy0_otg {
           rockchip,vbus-always-on;
   };
   ```

   注意：关闭充电检测功能后，同时也会关闭动态进入 suspend 的功能，以及关闭通知 DRD 控制器驱动切换到 Device mode。如果关闭充电检测功能后，Device mode 功能异常，建议将控制器的 `dr_mode` 属性配置为 `"peripheral"`。
5. 属性 `rockchip,dis-u2-susphy` 用于关闭 USB 2.0 PHY 动态进入 suspend mode 的功能，主要用于 USB 2.0 only 的方案，保持 USB2 PHY 输出时钟给 USB 控制器。

   ```
   &u2phy0_otg {
           rockchip,dis-u2-susphy;
   };
   ```

3.9.3 USB 3.0 Combo PHY DTS 属性配置说明

1. 属性 `rockchip,dis-u3otg0-port` 用于 USB3.0/SATA/PCIe Combo PHY0，当 PHY0 提供给 PCIe 或者 SATA 接口使用时，加上该属性禁用 USB 3.0 功能。
2. 属性 `rockchip,dis-u3otg1-port` 用于 USB3.0/SATA/PCIe Combo PHY1，当 PHY1 提供给 PCIe 或者 SATA 接口使用时，加上该属性禁用 USB 3.0 功能。

3.9.4 Android GKI 依赖属性配置

Android GKI 的 USB 控制器驱动与 USB 2.0 PHY 动态进入和退出 suspend mode 的机制存在兼容性问题，因此，需要关闭 USB 2.0 PHY 动态进入 suspend mode 的功能。参考 `arch/arm64/boot/dts/rockchip/rk3572-android.dtsi`

```
&u2phy0_otg {
        rockchip,dis-u2-susphy;
};
```

4. RK3572 USB DRD mode 切换命令

RK3572 SDK 支持通过软件方法，强制设置 USB DRD 控制器切换到 Host 或者 Device mode，而不受 USB 硬件电路的 DRD ID 电平或者 Type-C CC 的影响。

RK3572 内核支持两种 USB DRD mode 切换方式：

方式1.

```
#1.Force host mode
  echo host > /sys/devices/platform/soc/26054000.syscon/26054000.syscon:usb2-phy@0/otg_mode
#2.Force peripheral mode
  echo peripheral > /sys/devices/platform/soc/26054000.syscon/26054000.syscon:usb2-phy@0/otg_mode
```

方式2.

```
#1.Force host mode
  echo host > /sys/kernel/debug/usb/24000000.usb/mode
#2.Force peripheral mode
  echo device > /sys/kernel/debug/usb/24000000.usb/mode
```

Note.

方式 1 依赖于 USB DTS 的正确配置，只能用于非 Type-C 接口的硬件电路设计；方式 2 没有 USB DTS 配置限制。因此，在不确定软硬件是否正确适配时，推荐使用方式 2。

5. RK3572 USB 下载固件的功能配置

5.1 Rockusb3.0 下载工具配置

5.1.1 Windows 图形界面工具配置

版本要求：RKDevTool V3.37 或者更高版本、SocToolKit V2.5 或者更高版本、FactoryTool V1.9.1 或者更高版本。

驱动要求：V5.13 或者更新的驱动版本，如果驱动版本低于V5.13，可使用瑞芯微驱动助手 DriverAssitant_v5.13 安装驱动。

配置要求：配置工具的 config.ini 文件，使能 USB3 传输功能和配置等待USB连接的超时时间，根据不同的 Windows 工具，修改参考如下：

1. FactoryTool

配置 config.ini （文件放在 FactoryTool.exe 同一目录下）

```
#USB3_TRANSFER=TRUE时,设备端如果支持USB3传输，工具会采用USB3模式进行下载
USB3_TRANSFER=TRUE
#超时时间修改成5秒，减少MASKROM模式下USB 3枚举失败后降速到USB2的时间
ROCKUSB_TIMEOUT=5
```

2. SocToolKit

   配置 config.ini（文件放在工具的 bin\windows 目录下）

```
usb3_transfer_on=true
rkusb_timeout=5
```

3. RKDevTool

   配置 config.ini（文件放在 RKDevTool.exe 同一目录下）

```
#USB3_TRANSFER=TRUE时,设备端如果支持USB3传输，工具会采用USB3模式进行下载
USB3_TRANSFER=TRUE
#超时时间修改成5秒，减少MASKROM模式下USB 3枚举失败后降速到USB2的时间
ROCKUSB_TIMEOUT=5
```

5.1.2 Linux 命令行工具配置

版本要求：upgrade_tool V2.49 或者更高版本。

驱动要求：无

配置要求：配置 config.ini（文件放在 upgrade_tool 同一目录下）

```
usb3_transfer_on=true
rkusb_timeout=5
```

5.1.3 Mac 命令行工具配置

版本要求：upgrade_tool V2.44 或者更高版本。

驱动要求：无

配置要求：配置 config.ini（文件放在 upgrade_tool 同一目录下）

```
usb3_transfer_on=true
rkusb_timeout=5
```

5.2 Rockusb2.0 下载工具配置

如果产品硬件电路设计不支持 USB 3.0 下载功能，烧写工具必须关闭 USB 3.0 下载功能，否则会出现 Maskrom 下载固件失败的问题，修改方法是将章节Rockusb3.0 下载工具配置 中工具的配置参数 `USB3_TRANSFER` 或者 `usb3_transfer_on` 设置为 `false`。

5.3 Rockusb2.0 Kernel/U-Boot 配置

1. Kernel DTS 配置

   针对 Combo PHY0 未供电或者未使用的电路设计方案

   ```
   /* 关闭USB3.0/SATA/PCIe Combo PHY0 */
   &combphy0_psu {
           status = "disabled";
   };
   ```

   针对 Combo PHY0 提供给 PCIe/SATA 使用的电路设计方案

   ```
   /* 配置USB3.0/SATA/PCIe Combo PHY0 */
   &combphy0_psu {
           rockchip,dis-u3otg0-port;
           status = "okay";
   };
   ```

2. U-Boot 配置

   在 U-Boot 工程的 `configs/rk3572-usbplug_defconfig` 和 `configs/rk3572_defconfig` 中添加 `CONFIG_ROCKUSB_MAX_SPEED_HS`

6. RK3572 USB 2.0 低功耗唤醒系统的功能配置

6.1 RK3572 USB 2.0 唤醒功能描述

USB 2.0 唤醒相关场景如下：

- Device 模式：在待机状态下，通过 bvalid 上升沿来检测 host 接入，唤醒系统；
- Host 模式：在待机状态下，通过 DP/DM linestate 电平变化中断，来检测外设的接入或者 Remote wakeup，唤醒系统；
- OTG 模式：在待机状态下，通过 iddig 的电平变化中断，来检测 OTG 工作模式的切换，唤醒系统。

这几种唤醒方式，都需要 **USB 2.0 PHY 处于供电状态**，才能检测到 DP/DM/VBUS/ID 脚上的信号变化，并产生中断，送给 PMU，以唤醒系统。

Note. DP/DM linestate 电平变化的中断检测，与标准的 USB2 Remote wakeup 机制有区别。

相同点是，两者都要求主机侧的 USB 控制器支持 USB Remote Wakeup，USB 驱动通过 SET_FEATURE 和 CLEAR_FEATURE 的命令使能或者关闭 USB 外设的 Remote Wakeup 功能。在 USB bus 进入 suspend 之前，应使能 RemoteWakeup 特性。当 USB 外设要唤醒主机时，外设将先发起一个维持 3ms 的 Remote Wakeup 信号，通知主机系统启动 USB Resume 操作。

不同点是，标准的 USB 2.0 Remote wakeup 机制，要求主机收到模块的 Remote Wakeup 信号后，需要发送一个持续时间超过 20ms 应答信号，且主机必须在 Idle 状态开始的 3ms 内开始发送总线通信 (SOF token)，否则设备又将再次进入 Suspend，因此，要求待机时 USB 控制器保持供电。而 DP/DM linestate 电平变化的中断检测，则不需要主机端发送 Remote Wakeup 的应答信号，因此，可以允许在待机时关闭 USB 控制器的供电。

6.2 RK3572 USB 2.0 唤醒功能的硬件电路设计要求

二级待机的供电要求：

- USB2 PHY 相关的三路电源：USB_DRD_DVDD0V75、USB_DRD_AVDD1V8、USB_DRD_AVDD3V3 要保持供电；
- USB VBUS 需要保持对外供电；
- PMU 保持供电；
- 主控的 OSC 可以断电；
- 主控的 Logic 可以断电。

以 RK3572 EVB1 USB3.0 Host 接口的硬件电路参考设计为例：

1. USB2 PHY 相关的三路电源接合并到 PMU 的供电电源 VDD_0V75_S3/VCC_1V8_S3/VCC_3V3_S3

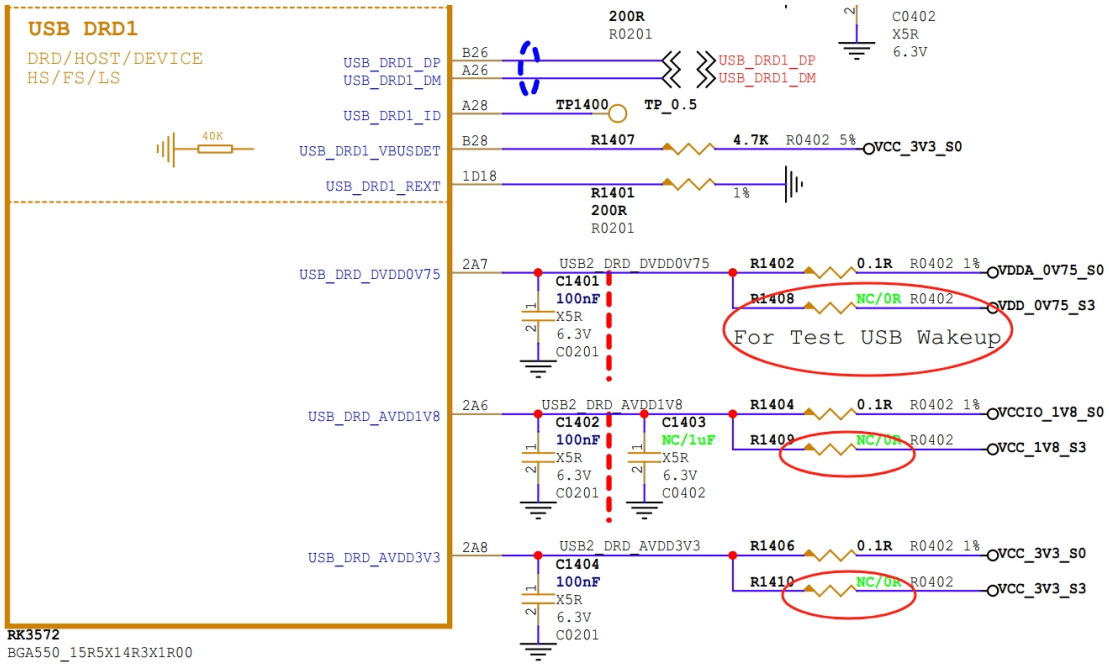


图 11 RK3572 USB 2.0 唤醒功能的 PHY 供电电源参考设计

2. USB VBUS 的供电由 VCC5V0_DEVICE_S0 修改到 VCC5v0_SYS_S5

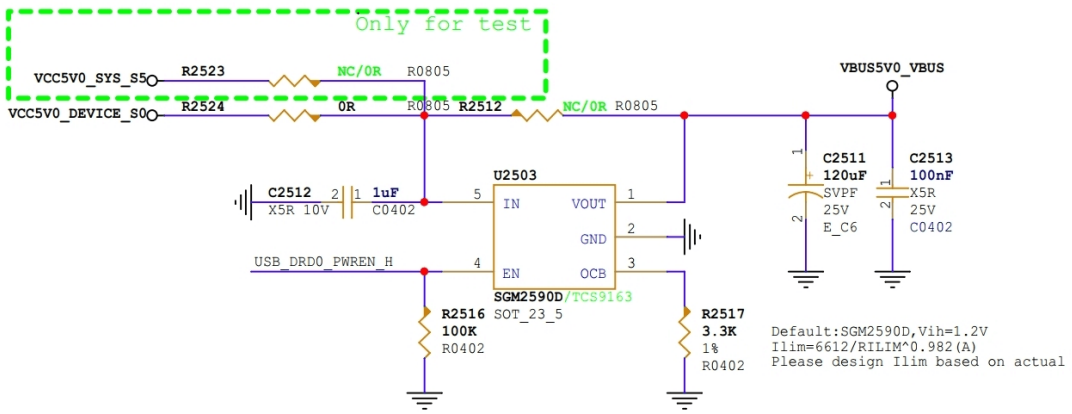


图 12 RK3572 USB 2.0 唤醒功能的 VBUS 电源参考设计

6.3 RK3572 USB 2.0 唤醒功能的软件配置

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3572.dtsi
b/arch/arm64/boot/dts/rockchip/rk3572.dtsi
index 8555ba5d126a..2c8d394ffa71 100644
--- a/arch/arm64/boot/dts/rockchip/rk3572.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3572.dtsi
@@ -1629,6 +1629,7 @@ rockchip_suspend: rockchip-suspend {
        rockchip,wakeup-config = <
            (0
            | RKPM_GPIO0_WKUP_EN
+           | RKPM_USB_WKUP_EN
            )
        >;
        rockchip,sleep-io-ret-config = <
```

7. RK3572 Type-C 控制器芯片支持列表

表 8 RK3572 Type-C 控制器芯片支持列表

Type-C 控制器芯片型号	**Linux-6.12**	说明
AW35615	支持	推荐优先使用，对 DisplayPort Alt Mode 兼容性好 软硬件完全兼容 FUSB302
ET7301B	支持	软硬件完全兼容 FUSB302note1
ET7303	支持	硬件兼容 FUSB302，软件驱动与 RT1711 高度相似 note2
FUSB302	支持	RK平台常用芯片note2
HUSB311	支持	RK3572 EVB1/2 默认使用的芯片 硬件兼容 FUSB302，但软件驱动不兼容note3
RT1711H	支持	硬件兼容 FUSB302，软件驱动与 ET7303 高度相似 note4
RK809C	支持	RK3572 Tablet 默认使用的芯片note5

更详细的 Type-C 控制器芯片支持列表信息，请参考 Redmine 平台 TYPE-C and PD support list：

https://redmine.rock-chips.com/documents/131

note1.

- Linux-6.12 采用 TCPM 软件框架和 TCPCI 协议，理论上可以兼容所有基于 TCPCI 标准设计的 Type-C 控制器芯片（如：ET7303/HUSB311/RT1711H）；

note2.

- 小封装的 ET7303 (据了解原厂目前没有提供大封装) 已经在 RK 平台验证通过。内核需要单独使能 CONFIG_TYPEC_ET7303。

- DTS 详细配置请参考章节 Type-C USB 3.0 Switch 全功能 DTS 配置 中 usbc0 节点，只需要修改该节点名字、reg 地址和 compatible 属性即可。

note3.

- FUSB302 可直接替换为 HUSB311。内核需要单独使用 CONFIG_TYPEC_HUSB311，DTS 配置注意点同上述 note2。

note4.

- RT1711H 硬件兼容 FUSB302/ET7303，同时软件驱动与 ET7303 高度相似。内核需要单独使能 CONFIG_TYPEC_RT1711H，DTS 配置注意点同上述 note2。

note5.

- RK809C 是支持 PD 功能的 PMIC 芯片，最高支持 12V 3A 充电，DTS 配置参考 Tablet DEMO 方案 Type-C USB 2.0 with PD DTS 配置。

8. 参考文档

1. 《Universal Serial Bus Type-C Cable and Connector Specification》
2. 《Universal Serial Bus Power Delivery Specification》
3. 《Rockchip_Developer_Guide_USB_CN》
4. 《Rockchip RK3572 Technical Reference Manual》